Unit 5: Structures, Dynamic Memory and File Handling

1. User-Defined Data Types in C

Definition

A data type specifies the type of value that a variable can store in a program. C provides basic data types such as int, float, char and double to store different kinds of data. Apart from these basic data types, C allows programmers to create their own data types according to program requirements. These are called user-defined data types, and they help organize and represent complex data in a meaningful way.

Types of User-Defined Data Types

C provides the following important user-defined data type mechanisms:

1. Structure (struct)
2. Union (union)
3. Enumeration (enum)
4. Type Definition (typedef)

1.1 Structure (struct)

A structure is a user-defined data type that groups multiple variables of different data types under a single name. Each variable inside a structure is called a member. For example, student information may contain a roll number, name and marks, which can be grouped into one structure. Each member has its own storage within the structure.

Example:

```
struct Student
{
   int rollno;
   char name[30];
   float marks;
};
```

Here, Student is a structure containing three members: rollno, name and marks.

1.2 Union (union)

A union is a user-defined data type that allows different members to share the same memory location. Unlike a structure, all union members use the same memory space. Therefore, a union generally stores a meaningful value in one member at a time. It is useful when different types of data need to use the same memory area at different times.

Example:

```
union Data
{
   int number;
   float price;
   char letter;
};
```

Here, Data contains three members, but they share the same memory space.

1.3 Enumeration (enum)

An enumeration is a user-defined data type used to define a set of named integer constants. It allows programmers to represent fixed choices using meaningful names instead of repeatedly using numeric values. By default, the first enumeration constant has the value zero, and subsequent constants increase by one. Enumeration makes programs easier to read and understand.

Example:

```
enum Week
{
   Monday, Tuesday, Wednesday, Thursday, Friday
};
```

By default:

- Monday = 0
- Tuesday = 1
- Wednesday = 2
- Thursday = 3
- Friday = 4

1.4 Type Definition (typedef)

typedef is a keyword used to create an alternative name for an existing data type. It does not create a completely new data type; instead, it provides a shorter or more meaningful name for an existing type. It can be used with basic data types, structures and other types. It makes declarations easier to write and improves code readability.

Example:

```
typedef unsigned int UINT;

UINT age = 20;
```

Here, UINT is an alternative name for unsigned int.

Difference Between User-Defined Data Types

Type	Purpose
Structure	Groups different data members, each with separate storage.
Union	Allows different members to share the same memory.
Enumeration	Defines named integer constants.
typedef	Creates an alternative name for an existing type.

Important point: User-defined data types allow programmers to organize data according to the requirements of a program.

2. Structure Declaration, Initialization and Member Access

Definition

A structure is a user-defined data type in C that combines different types of variables into a single unit. It is declared using the struct keyword followed by a structure name and its members. After declaring a structure, variables of that structure type can be created to store related information. Individual members of a structure can be accessed and modified using the dot (.) operator.

2.1 Structure Declaration

Structure declaration defines the structure name and the members that belong to it. The members can have different data types, such as integer, character and floating-point types. The declaration specifies the format of the structure but does not itself create a variable unless variables are declared along with it. The struct keyword is used to declare a structure.

Syntax:

```
struct structure_name
{
   data_type member1;
   data_type member2;
   data_type member3;
};
```

Example:

```
struct Student
{
   int rollno;
   char name[30];
   float marks;
};
```

2.2 Structure Variable Declaration

After declaring a structure, we can create variables using the structure name. These variables are used to store actual values for each member of the structure. Multiple variables of the same structure type can be declared. Each structure variable has its own separate set of members.

Syntax:

```
struct structure_name variable_name;
```

Example:

```
struct Student s1;
```

Here, s1 is a structure variable containing rollno, name and marks.

2.3 Structure Initialization

Structure initialization means assigning initial values to the members of a structure variable when it is declared. The values are generally provided inside curly braces in the same order as the members are declared. The values must be compatible with their respective member data types. This method allows all the required structure members to be initialized at the time of declaration.

Syntax:

```
struct structure_name variable = {value1, value2, value3};
```

Example:

```
struct Student s1 = {101, "Vamshi", 85.5};
```

Here:

- 101 is assigned to rollno.
- "Vamshi" is assigned to name.
- 85.5 is assigned to marks.

2.4 Structure Member Access

Structure members are accessed using the dot (.) operator. The structure variable name is written first, followed by a dot and the required member name. The dot operator allows programmers to read or modify individual member values. It is used whenever a structure variable is accessed directly.

Syntax:

```
variable_name.member_name;
```

Example:

```
s1.rollno = 101;
s1.marks = 90.5;
```

Complete C Program

Program to declare, initialize and access structure members.

```
#include <stdio.h>

struct Student
{
   int rollno;
   char name[30];
   float marks;
};

int main()
{
   struct Student s1 = {101, "Vamshi", 85.5};

   printf("Student Details\n");

   printf("Roll Number = %d\n", s1.rollno);
   printf("Name = %s\n", s1.name);
   printf("Marks = %.2f\n", s1.marks);

   return 0;
}
```

Output

```
Student Details
Roll Number = 101
```

```
Name = Vamshi
Marks = 85.50
```

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output header file.
2. `struct Student` declares a structure named `Student`.
3. `int rollno;` declares an integer member to store the student's roll number.
4. `char name[30];` declares a character array to store the student's name.
5. `float marks;` declares a floating-point member to store the student's marks.
6. `int main()` is the starting point of program execution.
7. `struct Student s1 = {101, "Vamshi", 85.5};` declares a structure variable and initializes all its members.
8. `s1.rollno` accesses the roll number member.
9. `s1.name` accesses the name member.
10. `s1.marks` accesses the marks member.
11. `printf()` displays the details of the student.
12. `return 0;` terminates the program successfully.

Important Points

- A structure is declared using the `struct` keyword.
- A structure can contain members of different data types.
- Structure members are accessed using the dot (.) operator.
- Each structure variable has its own separate member storage.
- A structure can contain arrays, pointers and other structures as members.

3. Arrays of Structures

Definition

An array of structures is a collection of structure variables of the same structure type stored under a single array name. It is used when we need to store information about multiple records having the same fields. Each element of the array represents one complete record and contains all the members declared in the structure. Arrays of structures make it easier to store, access and process large numbers of records using loops.

Example

Suppose we want to store the details of three students. Each student has a roll number, name and marks.

Instead of declaring three separate structure variables, we can create an array of structures.

```
struct Student
{
   int rollno;
   char name[30];
   float marks;
};

struct Student s[3];
```

Here:

- Student is the structure name.
- s[3] creates an array of three structure variables.
- s[0] represents the first student.
- s[1] represents the second student.
- s[2] represents the third student.

Syntax

```
struct structure_name
{
   data_type member1;
   data_type member2;
};

struct structure_name array_name[size];
```

Accessing Array Structure Members

The members of an array of structures are accessed using the array index and dot operator.

Syntax:

```
array_name[index].member_name;
```

Example:

```
s[0].rollno = 101;
s[0].marks = 85.5;
```

Here, the roll number and marks of the first student are assigned.

C Program

Program to store and display details of three students using an array of structures.

```
#include <stdio.h>

struct Student
{
   int rollno;
   char name[30];
   float marks;
};

int main()
{
   struct Student s[3];
   int i;

   for(i = 0; i < 3; i++)
   {
      printf("Enter roll number: ");
      scanf("%d", &s[i].rollno);

      printf("Enter name: ");
      scanf("%29s", s[i].name);
```

```
        printf("Enter marks: ");
        scanf("%f", &s[i].marks);
    }

    printf("\nStudent Details\n");

    for(i = 0; i < 3; i++)
    {
        printf("\nRoll Number = %d", s[i].rollno);
        printf("\nName = %s", s[i].name);
        printf("\nMarks = %.2f\n", s[i].marks);
    }

    return 0;
}
```

Output

```
Enter roll number: 101
Enter name: Ravi
Enter marks: 85

Enter roll number: 102
Enter name: Priya
Enter marks: 90

Enter roll number: 103
Enter name: Kumar
Enter marks: 78

Student Details

Roll Number = 101
Name = Ravi
Marks = 85.00

Roll Number = 102
Name = Priya
Marks = 90.00

Roll Number = 103
Name = Kumar
Marks = 78.00
```

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output header file.
2. `struct Student` declares a structure named `Student`.
3. `int rollno;` stores the student's roll number.
4. `char name[30];` stores the student's name.
5. `float marks;` stores the student's marks.
6. `struct Student s[3];` declares an array containing three structure variables.
7. `int i;` declares a loop control variable.
8. The first `for` loop executes three times to accept the details of three students.
9. `s[i].rollno` accesses the roll number member of the current student.
10. `s[i].name` accesses the name member of the current student.
11. `s[i].marks` accesses the marks member of the current student.
12. The second `for` loop displays the details of all three students.
13. `return 0;` terminates the program successfully.

Advantages of Arrays of Structures

1. Multiple Records: It stores details of many students, employees or products using one array.
2. Easy Access: Each record can be accessed using its array index.
3. Less Code: Loops can be used to read and display multiple records.
4. Easy Management: Records of the same type can be handled together.

Important point: An array of structures stores multiple records of the same structure type under one array name.

4. Nested Structures

Definition

A nested structure is a structure declared as a member inside another structure. It allows programmers to group related information into smaller structures and combine them into a larger structure. For example, student details may contain personal information and address information, which can be maintained using separate structures. Nested structures help organize complex data and make programs easier to understand and maintain.

Example

Consider a student record containing:

- Roll number
- Student name
- City
- PIN code

We can create one structure for address details and another structure for student details. The address structure can be included as a member of the student structure.

Syntax

```
struct Structure1
{
   data_type member1;
   data_type member2;
};

struct Structure2
{
   data_type member3;
   struct Structure1 variable;
};
```

Example Structure Declaration

```
struct Address
{
   char city[30];
   int pincode;
};

struct Student
{
```

```
    int rollno;
    char name[30];
    struct Address addr;
};
```

Here:

- Address is the inner structure.
- Student is the outer structure.
- addr is a structure variable declared inside Student.

Accessing Nested Structure Members

The dot operator is used to access members of a nested structure. First, we write the outer structure variable name, followed by a dot and the inner structure variable name. Another dot is used to access the required member of the inner structure.

Syntax:

```
outer_variable.inner_variable.member_name;
```

Example:

```
s1.addr.city = "Gangavathi";
```

C Program

Program to demonstrate nested structures.

```
#include <stdio.h>

struct Address
{
    char city[30];
    int pincode;
};

struct Student
{
    int rollno;
    char name[30];
    struct Address addr;
};

int main()
{
    struct Student s1;

    printf("Enter roll number: ");
    scanf("%d", &s1.rollno);

    printf("Enter name: ");
    scanf("%29s", s1.name);

    printf("Enter city: ");
    scanf("%29s", s1.addr.city);

    printf("Enter pincode: ");
    scanf("%d", &s1.addr.pincode);
```

```
    printf("\nStudent Details\n");

    printf("Roll Number = %d\n", s1.rollno);
    printf("Name = %s\n", s1.name);
    printf("City = %s\n", s1.addr.city);
    printf("Pincode = %d\n", s1.addr.pincode);

    return 0;
}
```

Output

```
Enter roll number: 101
Enter name: Vamshi
Enter city: Gangavathi
Enter pincode: 583227

Student Details
Roll Number = 101
Name = Vamshi
City = Gangavathi
Pincode = 583227
```

Explanation of the Program

1. `struct Address` declares a structure to store address information.
2. `char city[30];` stores the city name.
3. `int pincode;` stores the PIN code.
4. `struct Student` declares another structure to store student information.
5. `struct Address addr;` includes the `Address` structure as a member of `Student`.
6. `struct Student s1;` declares a variable of the outer structure.
7. `s1.rollno` accesses the roll number member.
8. `s1.name` accesses the student name.
9. `s1.addr.city` accesses the city member inside the nested structure.
10. `s1.addr.pincode` accesses the PIN code member inside the nested structure.
11. `printf()` displays all the student details.
12. `return 0;` terminates the program successfully.

Advantages of Nested Structures

1. They organize complex information into smaller groups.
2. They improve program readability.
3. They allow one structure to be reused inside another structure.
4. They make it easier to represent related information.

Important point: A nested structure is a structure that contains another structure as one of its members.

5. Passing Structures to Functions

Definition

Passing structures to functions means sending the data stored in a structure variable to a function for processing. In C programming, a complete structure can be passed to a function as an argument. The function can receive the structure through a structure parameter and access its individual members. Structures can also be passed using pointers, which allows the function to work with the original structure without copying the entire structure value.

Methods of Passing Structures to Functions

1. Passing an individual structure member.
2. Passing the complete structure variable.
3. Passing the structure using a pointer.

5.1 Passing Individual Structure Members

In this method, individual members of a structure are passed separately as arguments to a function. The function receives each value through its corresponding parameter. It is similar to passing ordinary variables to a function. This method is useful when the function needs only a few members of the structure.

Example:

```
display(s1.rollno, s1.marks);
```

5.2 Passing the Complete Structure

In this method, the complete structure variable is passed to a function as an argument. The function receives a copy of the structure and can access its members using the dot operator. Changes made to the received copy do not affect the original structure variable. This method is useful when a function needs all or most of the structure members.

Syntax:

```
function_name(structure_variable);
```

C Program

Program to pass a structure to a function.

```
#include <stdio.h>

struct Student
{
   int rollno;
   char name[30];
   float marks;
};

void display(struct Student s)
{
   printf("\nStudent Details\n");

   printf("Roll Number = %d\n", s.rollno);
   printf("Name = %s\n", s.name);
```

```
    printf("Marks = %.2f\n", s.marks);
}

int main()
{
    struct Student s1 = {101, "Vamshi", 85.5};

    display(s1);

    return 0;
}
```

Output

```
Student Details
Roll Number = 101
Name = Vamshi
Marks = 85.50
```

Explanation of the Program

1. `struct Student` declares a structure containing roll number, name and marks.
2. `void display(struct Student s)` defines a function that accepts a structure variable as an argument.
3. The parameter `s` receives a copy of the structure variable passed to the function.
4. `s.rollno` accesses the roll number member.
5. `s.name` accesses the name member.
6. `s.marks` accesses the marks member.
7. `int main()` is the starting point of execution.
8. `struct Student s1 = {101, "Vamshi", 85.5};` creates and initializes a structure variable.
9. `display(s1);` passes the complete structure variable to the display function.
10. The display function prints the received student details.
11. `return 0;` terminates the program.

5.3 Passing Structure Using a Pointer

In this method, the address of a structure variable is passed to a function instead of passing the entire structure value. The function receives the address through a structure pointer and accesses the members using the arrow (->) operator. This method avoids copying the complete structure and allows the function to modify the original structure. It is useful when working with large structures.

Syntax:

```
function_name(&structure_variable);
```

Function definition:

```
void display(struct Student *s)
{
    printf("%d", s->rollno);
}
```

Example:

```
#include <stdio.h>

struct Student
{
    int rollno;
```

```
    char name[30];
};

void display(struct Student *s)
{
    printf("Roll Number = %d\n", s->rollno);
    printf("Name = %s\n", s->name);
}

int main()
{
    struct Student s1 = {101, "Ravi"};

    display(&s1);

    return 0;
}
```

Output:

```
Roll Number = 101
Name = Ravi
```

Difference Between Dot and Arrow Operators

Dot (.) operator	**Arrow (->) operator**
Used with a structure variable.	Used with a pointer to a structure.
Accesses a member directly.	Accesses a member through a pointer.
Example: s1.rollno	Example: p->rollno

Important point: A structure can be passed to a function by value or through a pointer. Passing a pointer is generally more efficient for large structures.

6. Unions and Comparison of Structures and Unions

Definition of Union

A union is a user-defined data type in C that allows different members to share the same memory location. It is declared using the union keyword, followed by a union name and its members. All members of a union begin at the same memory location, so writing to one member can change the value previously stored through another member. The size of a union is sufficient to store its largest member, including any alignment requirements.

Example

```
union Data
{
   int number;
   float price;
   char letter;
};
```

Here:

- number is an integer member.
- price is a floating-point member.
- letter is a character member.
- All three members share the same memory space.

Syntax

```
union union_name
{
   data_type member1;
   data_type member2;
   data_type member3;
};
```

Declaring a Union Variable

```
union Data d1;
```

Here, d1 is a variable of union type Data.

Accessing Union Members

Union members are accessed using the dot (.) operator, just like structure members.

Syntax:

```
variable_name.member_name;
```

Example:

```
d1.number = 100;
printf("%d", d1.number);
```

C Program

Program to demonstrate a union.

```
#include <stdio.h>

union Data
{
   int number;
   float price;
   char letter;
};

int main()
{
   union Data d1;

   d1.number = 100;

   printf("Number = %d\n", d1.number);

   d1.price = 25.5;

   printf("Price = %.2f\n", d1.price);

   d1.letter = 'A';

   printf("Letter = %c\n", d1.letter);

   return 0;
}
```

Output

```
Number = 100
Price = 25.50
Letter = A
```

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output header file.
2. `union Data` declares a union named `Data`.
3. `int number;` declares an integer member.
4. `float price;` declares a floating-point member.
5. `char letter;` declares a character member.
6. `union Data d1;` creates a union variable.
7. `d1.number = 100;` stores 100 in the union's shared memory.
8. `printf()` displays the integer value while `number` is the active member.
9. `d1.price = 25.5;` stores a floating-point value in the same shared memory, replacing the previously stored value.
10. `d1.letter = 'A';` stores a character in the shared memory.
11. `printf()` displays the character.
12. `return 0;` terminates the program successfully.

Important: Only the most recently written member should normally be read. Reading another member after writing a different member does not provide a portable way to retrieve the earlier value.

Difference Between Structure and Union

A structure and a union are both user-defined data types used to group different types of members. However, they differ in how memory is allocated to their members. In a structure, each member has its own storage, so all members can hold their values at the same time. In a union, all members share the same memory location, so storing a new member value can overwrite the previous member's stored representation.

Structure (struct)	**Union (union)**
Each member has separate storage within the structure.	All members share the same storage.
All members can hold their values simultaneously.	Only one member's value should normally be used at a time.
Size is at least the sum of member sizes, with possible padding.	Size is at least the size of its largest member, with possible padding.
Modifying one member does not change another member's value.	Writing to one member can replace the previously stored member value.
Uses the struct keyword.	Uses the union keyword.
Suitable for storing complete records.	Suitable when different data types use the same memory at different times.

Example to Understand Memory Usage

Consider the following declarations:

```
struct Example1
{
   int a;
   float b;
   char c;
};

union Example2
{
   int a;
   float b;
   char c;
};
```

Assuming int and float each occupy 4 bytes and char occupies 1 byte:

- The structure needs storage for all three members, plus any padding required for alignment.
- The union needs enough storage for its largest member, plus any alignment padding.

For this example, the union will commonly occupy 4 bytes, while the structure will commonly occupy 12 bytes. Actual sizes depend on the implementation and alignment rules.

Important point: Structures allocate separate storage for their members, whereas unions share storage among all their members.

7. Enumeration (enum) and typedef

7.1 Enumeration (enum)

Definition

Enumeration is a user-defined data type in C that consists of a group of named integer constants. It is declared using the enum keyword and is useful when a variable needs to represent one value from a fixed set of choices. By default, the first enumerator is assigned the value zero, and each following enumerator receives the next integer value. Enumeration makes programs easier to read because meaningful names can be used instead of numeric constants.

Example

Consider the days of a week. Instead of using numbers to represent the days, we can create an enumeration with meaningful names.

```
enum Week
{
   Monday,
   Tuesday,
   Wednesday,
   Thursday,
   Friday,
   Saturday,
   Sunday
};
```

By default, the values are:

Enumerator	Integer value
Monday	0
Tuesday	1
Wednesday	2
Thursday	3
Friday	4
Saturday	5
Sunday	6

Syntax

```
enum enum_name
{
   constant1,
   constant2,
   constant3
};
```

Declaring an Enumeration Variable

```
enum Week today;
```

C Program

Program to demonstrate enumeration.

```
#include <stdio.h>

enum Week
{
   Monday = 1,
   Tuesday,
   Wednesday,
   Thursday,
   Friday
};

int main()
{
   enum Week today;

   today = Wednesday;

   printf("Day number = %d", today);

   return 0;
}
```

Output

```
Day number = 3
```

Explanation of the Program

1. `enum Week` declares an enumeration named `Week`.
2. `Monday = 1` assigns the integer value 1 to Monday.
3. `Tuesday` automatically receives the value 2.
4. `Wednesday` automatically receives the value 3.
5. `Thursday` and `Friday` receive the values 4 and 5.
6. `enum Week today;` declares a variable named `today` of the enumeration type.
7. `today = Wednesday;` assigns the enumeration value of Wednesday to the variable.
8. `printf()` displays the integer value 3.
9. `return 0;` terminates the program successfully.

Advantages of Enumeration

1. Makes programs easier to read and understand.
2. Uses meaningful names instead of numeric values.
3. Helps represent a fixed set of choices.
4. Makes programs easier to maintain.

7.2 Type Definition (typedef)

Definition

typedef is a keyword in C used to create an alternative name for an existing data type. It allows programmers to use shorter and more meaningful names instead of writing long or complex type declarations repeatedly. It can be used with basic data types, structures, unions and other types. It improves program readability and simplifies variable declarations.

Example

Without typedef:

```
unsigned int age;
```

With typedef:

```
typedef unsigned int UINT;

UINT age;
```

Here, UINT is an alternative name for unsigned int.

Syntax

```
typedef existing_data_type new_name;
```

Example 1: Using typedef with Basic Data Types

```
#include <stdio.h>

typedef int NUMBER;

int main()
{
   NUMBER a = 10;

   printf("Value = %d", a);

   return 0;
}
```

Output:

```
Value = 10
```

Explanation:

- typedef int NUMBER; creates NUMBER as an alternative name for int.
- NUMBER a = 10; declares an integer variable using the new name.
- printf() displays the value of a.

Example 2: Using typedef with Structures

Normally, a structure variable is declared using the structure name with the struct keyword.

```
struct Student
{
   int rollno;
   char name[30];
};
```

A variable is declared as:

```
struct Student s1;
```

Using typedef, we can simplify this declaration.

Program:

```
#include <stdio.h>

typedef struct
{
   int rollno;
   char name[30];
} Student;

int main()
{
   Student s1 = {101, "Vamshi"};

   printf("Roll Number = %d\n", s1.rollno);
   printf("Name = %s", s1.name);

   return 0;
}
```

Output:

```
Roll Number = 101
Name = Vamshi
```

Explanation of the Program

1. typedef struct defines a structure and creates an alternative type name.
2. int rollno; declares a member to store the roll number.
3. char name[30]; declares a member to store the student's name.
4. Student becomes the alternative name for this structure type.
5. Student s1 = {101, "Vamshi"}; declares and initializes a structure variable without writing the struct keyword.
6. s1.rollno accesses the roll number.
7. s1.name accesses the student's name.
8. printf() displays the structure members.

Difference Between enum and typedef

Enumeration (enum)	Type Definition (typedef)
Defines a set of named integer constants.	Creates an alternative name for an existing type.
Uses the enum keyword.	Uses the typedef keyword.
Useful for representing fixed choices.	Useful for simplifying type declarations.
Example: enum Week {Mon, Tue};	Example: typedef int NUMBER;

Important points:

- enum creates named integer constants.
- typedef provides an alternative name for a data type.
- Both improve program readability.
- typedef with structures avoids repeatedly writing the struct keyword.

8. Introduction to Dynamic Memory Allocation

Definition

Dynamic memory allocation is the process of allocating memory during program execution according to the requirements of a program. Normally, when we declare an array, its size is specified before the program runs, and the memory requirement is fixed. However, sometimes we do not know how much memory is needed in advance. Dynamic memory allocation allows us to request memory while the program is running and use it according to the input or requirements. In C, dynamic memory allocation is performed using functions available in the stdlib.h header file.

Why Do We Need Dynamic Memory Allocation?

Consider a program that stores student marks.

- If we declare int marks[100];, memory is reserved for 100 integers.
- But what if there are only 10 students? The remaining array capacity is unused.
- What if there are 500 students? The array cannot store all the values.

Dynamic memory allocation helps us request memory based on the actual number of students.

Example

Suppose the user enters the number of students as 5.

The program can request memory for exactly five integer values during execution.

```
int *marks;

marks = malloc(5 * sizeof(int));
```

Here:

- int *marks declares an integer pointer.
- malloc() requests memory.
- 5 represents the number of integer elements.
- sizeof(int) gives the size of one integer.
- marks stores the address of the allocated memory.

Dynamic Memory Allocation Functions

C provides four important functions for dynamic memory allocation.

Function	Purpose
malloc()	Allocates a block of memory.
calloc()	Allocates memory for multiple elements and initializes all bytes to zero.
realloc()	Changes the size of previously allocated memory.
free()	Releases dynamically allocated memory.

Header File

All four functions are declared in:

```
#include <stdlib.h>
```

General Syntax

```
pointer = malloc(size);
pointer = calloc(number, size);
pointer = realloc(pointer, new_size);
free(pointer);
```

Difference Between Static and Dynamic Memory Allocation

Static/Fixed Allocation	Dynamic Allocation
Memory size is generally specified before execution.	Memory is requested during execution.
Array capacity is fixed after declaration.	Allocated memory can be resized using realloc().
Ordinary local arrays usually have automatic storage duration.	Dynamically allocated memory remains allocated until released.
Example: int a[10];	Example: int *p = malloc(10 * sizeof(int));

Important point: Dynamic memory allocation allows a program to request memory during execution according to its actual requirements.

9. malloc() Function

Definition

malloc() stands for Memory Allocation. It is a standard library function in C used to allocate a specified number of bytes in memory during program execution. It allocates one continuous block of memory and returns a pointer to the beginning of that block. The allocated memory is not automatically initialized, so it contains indeterminate values. If memory allocation fails, malloc() returns a null pointer (NULL).

Syntax

```
pointer = malloc(number_of_bytes);
```

Example:

```
int *p;

p = malloc(5 * sizeof(int));
```

Explanation:

- int *p; declares an integer pointer.
- sizeof(int) calculates the memory required for one integer.
- 5 * sizeof(int) calculates the memory required for five integers.
- malloc() allocates the required memory.
- p stores the address of the allocated memory.

Working of malloc()

Suppose an integer requires 4 bytes on the system being used.

If we write:

```
p = malloc(5 * sizeof(int));
```

The requested memory is:

[5 times 4 = 20 text{ bytes}]

Memory allocated for five integers

p[0]

Integer

p[1]

Integer

p[2]

Integer

p[3]

Integer

p[4]

Integer
One continuous block of memory for five integer elements. The actual byte size depends on the system.

C Program: Program to allocate memory using malloc() and display array elements.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int *p;
   int n, i;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   if(n <= 0)
   {
      printf("Invalid size");
      return 1;
   }

   p = malloc(n * sizeof(int));

   if(p == NULL)
   {
      printf("Memory allocation failed");
      return 1;
   }

   printf("Enter %d numbers:\n", n);

   for(i = 0; i < n; i++)
   {
      scanf("%d", &p[i]);
   }

   printf("Entered numbers are:\n");

   for(i = 0; i < n; i++)
   {
      printf("%d\n", p[i]);
   }

   free(p);

   return 0;
}
```

Output

```
Enter number of elements: 3
Enter 3 numbers:
10
20
30
Entered numbers are:
10
20
30
```

Explanation of the Program

1. #include <stdio.h> includes input-output functions.
2. #include <stdlib.h> includes memory allocation functions.
3. int *p; declares an integer pointer to store the address of allocated memory.
4. int n, i; declares variables to store the number of elements and loop counter.
5. scanf("%d", &n); accepts the number of elements from the user.
6. if(n <= 0) checks whether the entered size is valid.
7. p = malloc(n * sizeof(int)); requests memory for the required number of integers.
8. if(p == NULL) checks whether memory allocation was successful.
9. The first for loop accepts the integer values from the user.
10. p[i] accesses each dynamically allocated integer element.
11. The second for loop displays all the entered values.
12. free(p); releases the allocated memory.
13. return 0; terminates the program successfully.

Important Points

- malloc() allocates memory during execution.
- It returns a pointer to the allocated memory.
- The allocated memory is not initialized.
- It returns NULL if allocation fails.
- The allocated memory should be released using free() when it is no longer required.

10. calloc() Function

Definition

calloc() stands for Contiguous Allocation. It is a standard library function used to allocate memory for multiple elements during program execution. Unlike malloc(), it accepts two arguments: the number of elements and the size of each element. It allocates one continuous block of memory and initializes all allocated bytes to zero. If memory allocation fails, it returns NULL.

Syntax

```
pointer = calloc(number_of_elements, size_of_each_element);
```

Example:

```
int *p;

p = calloc(5, sizeof(int));
```

Explanation:

- 5 represents the number of elements.
- sizeof(int) represents the size of each integer.
- calloc() allocates memory for five integers.
- All allocated bytes are initialized to zero.

Difference Between malloc() and calloc()

malloc()	calloc()
Stands for Memory Allocation.	Stands for Contiguous Allocation.
Takes one argument.	Takes two arguments.
Accepts the total number of bytes.	Accepts the number of elements and size of each element.
Does not initialize allocated memory.	Initializes all allocated bytes to zero.
Example: malloc(5 * sizeof(int))	Example: calloc(5, sizeof(int))

C Program

Program to allocate memory using calloc() and display initial values.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int *p;
   int n, i;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   if(n <= 0)
   {
      printf("Invalid size");
      return 1;
   }

   p = calloc(n, sizeof(int));

   if(p == NULL)
   {
      printf("Memory allocation failed");
      return 1;
   }

   printf("Initial values:\n");

   for(i = 0; i < n; i++)
   {
      printf("%d\n", p[i]);
   }

   free(p);

   return 0;
}
```

Output

```
Enter number of elements: 5
Initial values:
0
0
0
0
0
```

Explanation of the Program

1. `#include <stdio.h>` includes standard input-output functions.
2. `#include <stdlib.h>` includes dynamic memory allocation functions.
3. `int *p;` declares an integer pointer.
4. `int n, i;` declares variables to store the number of elements and loop counter.
5. `scanf("%d", &n);` accepts the number of elements from the user.
6. `if(n <= 0)` checks whether the entered size is valid.
7. `p = calloc(n, sizeof(int));` allocates memory for the required number of integers.
8. `if(p == NULL)` checks whether memory allocation failed.
9. `printf("Initial values:\n");` displays a heading.
10. The `for` loop accesses each allocated element.
11. `printf("%d\n", p[i]);` displays the value of each element. All bytes have been initialized to zero, so each `int` element has the value zero.
12. `free(p);` releases the allocated memory.
13. `return 0;` terminates the program successfully.

Important Points

- `calloc()` allocates memory for multiple elements.
- It accepts two arguments.
- It initializes all allocated bytes to zero.
- It returns `NULL` if memory allocation fails.
- The allocated memory should be released using `free()`.

11. realloc() Function

Definition

`realloc()` stands for Reallocation. It is a standard library function used to change the size of a previously allocated memory block. It can increase or decrease the amount of dynamically allocated memory according to the program's requirements. The function may keep the memory at the same location or move it to a different location, while preserving the existing contents up to the smaller of the old and new sizes. If reallocation fails for a nonzero requested size, the original memory block remains allocated and unchanged.

Why Do We Need realloc()?

Suppose a program initially allocates memory for three students.

```
int *p;

p = malloc(3 * sizeof(int));
```

Later, the user wants to store five students.

Instead of creating an entirely new array manually, we can request a larger memory block using `realloc()`.

```
p = realloc(p, 5 * sizeof(int));
```

However, assigning the result directly to `p` can cause loss of the original pointer if reallocation fails. It is safer to use a temporary pointer.

Syntax

```
new_pointer = realloc(old_pointer, new_size);
```

Example:

```
int *temp;

temp = realloc(p, 5 * sizeof(int));

if(temp != NULL)
{
   p = temp;
}
```

Explanation:

- p stores the address of the original memory block.
- realloc() requests a new size sufficient for five integers.
- temp receives the returned pointer.
- If temp is not NULL, p is updated to the new address.

Working of realloc()

Initially, memory is allocated for three elements.

Before reallocation

10

20

30

Memory for three integers.
realloc()
Request memory for five integers

After reallocation

10

20

30

New

New

The first three values are preserved. Newly added memory has indeterminate values until initialized.

C Program

Program to increase memory using realloc().

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int *p, *temp;
   int i;

   p = malloc(3 * sizeof(int));

   if(p == NULL)
   {
      printf("Memory allocation failed");
      return 1;
   }

   printf("Enter 3 numbers:\n");

   for(i = 0; i < 3; i++)
   {
      scanf("%d", &p[i]);
   }

   temp = realloc(p, 5 * sizeof(int));

   if(temp == NULL)
   {
      printf("Reallocation failed");
      free(p);
      return 1;
   }

   p = temp;

   printf("Enter 2 more numbers:\n");

   for(i = 3; i < 5; i++)
   {
      scanf("%d", &p[i]);
   }

   printf("All numbers are:\n");

   for(i = 0; i < 5; i++)
   {
      printf("%d\n", p[i]);
   }

   free(p);

   return 0;
}
```

Output

```
Enter 3 numbers:
10
20
30
Enter 2 more numbers:
40
50
All numbers are:
10
20
30
40
50
```

Explanation of the Program

1. `#include <stdio.h>` includes input-output functions.
2. `#include <stdlib.h>` includes memory allocation functions.
3. `int *p, *temp;` declares two integer pointers.
4. `p = malloc(3 * sizeof(int));` allocates memory for three integers.
5. `if(p == NULL)` checks whether the initial allocation was successful.
6. The first `for` loop accepts three integer values.
7. `temp = realloc(p, 5 * sizeof(int));` requests a memory block large enough for five integers.
8. `if(temp == NULL)` checks whether reallocation failed. The original pointer `p` is still valid when a nonzero reallocation request fails.
9. `p = temp;` updates the original pointer after successful reallocation.
10. The second `for` loop accepts two additional values in positions 3 and 4.
11. The third `for` loop displays all five elements.
12. `free(p);` releases the allocated memory.
13. `return 0;` terminates the program successfully.

Important Points

- `realloc()` changes the size of previously allocated memory.
- It can increase or decrease memory size.
- It preserves existing data up to the smaller of the old and new sizes.
- It may move the memory block to a new location.
- Use a temporary pointer to avoid losing the original memory address if reallocation fails.
- Newly added memory is not initialized.

12. free() Function

Definition

free() is a standard library function used to release dynamically allocated memory when it is no longer required by the program. Memory allocated using malloc(), calloc() or realloc() remains allocated until it is released or the program terminates. If allocated memory is not released when it is no longer needed, it can contribute to memory leaks. The free() function helps manage memory efficiently by returning the allocated block to the memory allocator.

Why Do We Need free()?

Suppose a program allocates memory for 100 integers using malloc().

```
int *p;

p = malloc(100 * sizeof(int));
```

After using the allocated memory, we should release it.

```
free(p);
```

This tells the memory allocator that the allocated block is no longer needed.

Syntax

```
free(pointer);
```

Example:

```
int *p;

p = malloc(5 * sizeof(int));

free(p);

p = NULL;
```

Explanation:

- malloc() allocates memory.
- free(p) releases the allocated memory.
- p = NULL ensures that the pointer no longer holds the old address.

C Program

Program to demonstrate dynamic memory allocation and deallocation.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int *p;

   p = malloc(sizeof(int));

   if(p == NULL)
   {
      printf("Memory allocation failed");
      return 1;
   }

   *p = 100;

   printf("Value = %d\n", *p);

   free(p);

   p = NULL;

   printf("Memory released successfully");

   return 0;
}
```

Output

```
Value = 100
Memory released successfully
```

Explanation of the Program

1. `#include <stdio.h>` includes standard input-output functions.
2. `#include <stdlib.h>` includes memory allocation and deallocation functions.
3. `int *p;` declares an integer pointer.
4. `p = malloc(sizeof(int));` allocates memory for one integer.
5. `if(p == NULL)` checks whether memory allocation was successful.
6. `*p = 100;` stores the value 100 in the allocated memory.
7. `printf()` displays the stored value.
8. `free(p);` releases the dynamically allocated memory.
9. `p = NULL;` assigns a null pointer to `p` to avoid leaving it with the old address.
10. `printf()` displays a confirmation message.
11. `return 0;` terminates the program successfully.

Important Points

- `free()` releases dynamically allocated memory.
- It can be used with memory allocated by `malloc()`, `calloc()` or `realloc()`.
- A pointer should not be dereferenced after its memory has been freed.
- Freeing the same allocated block more than once causes undefined behavior.
- Assigning `NULL` after freeing a pointer helps prevent accidental use of that pointer.

Difference Between All Four Dynamic Memory Allocation Functions

Function	Purpose	Initialization
malloc()	Allocates a specified number of bytes.	No initialization
calloc()	Allocates memory for multiple elements.	All bytes initialized to zero
realloc()	Changes the size of allocated memory.	Existing contents preserved up to the smaller size; new memory uninitialized
free()	Releases allocated memory.	Not applicable

Complete Memory Allocation Process

1. malloc() / calloc()

Allocate memory

2. Use allocated memory

Read, write and process values

3. realloc() (if required)

Increase or decrease memory size

4. free()

Release allocated memory

Advantages of Dynamic Memory Allocation

1. Flexible Memory Usage: Memory can be requested according to actual requirements.
2. Efficient Memory Management: Programs can avoid reserving unnecessarily large fixed arrays.
3. Resizable Memory: realloc() allows the allocated memory size to be changed.
4. Useful for Data Structures: Dynamic memory is important for linked lists, trees and other dynamic data structures.
5. Runtime Allocation: Memory can be allocated after the user enters the required number of elements.

Important Examination Points

- Header file for dynamic memory allocation: stdlib.h.
- malloc() allocates memory without initialization.
- calloc() allocates memory and initializes all bytes to zero.
- realloc() changes the size of an existing allocated block.
- free() releases dynamically allocated memory.
- NULL indicates that a memory allocation request failed.

13. Need for Files in C

Definition

A file is a collection of data stored permanently on a storage device such as a hard disk or SSD. In C programming, file handling is used to store information so that it can be accessed even after the program has finished execution. Normally, data stored in variables is temporary and is lost when the program terminates. Files solve this problem by allowing programs to write, read, update and manage information whenever required.

Why Do We Need Files?

Consider a college management program that stores student information.

Suppose we enter the following details:

- Roll Number: 101
- Name: Ravi
- Marks: 85

If we store these details only in ordinary variables, the data will be lost when the program terminates.

But if we store the details in a file, they remain available even after closing the program.

Example

Without File Handling:

1. The user enters student details.
2. The program stores the details in variables.
3. The program displays the details.
4. The program terminates.
5. The data stored in ordinary variables is lost.

With File Handling:

1. The user enters student details.
2. The program writes the details into a file.
3. The program terminates.
4. The data remains stored in the file.
5. The program can open the file later and retrieve the details.

Types of Files in C

C mainly works with two broad categories of files:

1. Text Files

A text file stores data in the form of readable characters. The information can be opened and viewed using a text editor such as Notepad or TextEdit. Numbers and other values are represented as characters in the file. Text files are useful for storing simple information such as student details, employee records and reports.

Example:

101 Ravi 85
102 Priya 90
103 Kumar 78

2. Binary Files

A binary file stores data as bytes in a format intended for program processing rather than ordinary text reading. The contents are generally not directly understandable using a normal text editor. Binary files are commonly used for storing structured data efficiently. They are useful when programs need to save and retrieve data in a compact binary representation.

Example: A file containing the raw binary representation of student records.

Difference Between Text Files and Binary Files

Text File	Binary File
Stores data as readable characters.	Stores data as bytes.
Can generally be opened in a text editor.	Usually requires a suitable program to interpret the data.
Numbers are stored in textual form.	Data can be stored in its binary representation.
Useful for simple records and reports.	Useful for compact data storage and structured binary information.
Example: .txt	Example: .dat, .bin

Advantages of File Handling

1. Permanent Storage: Data remains stored even after the program terminates.
2. Data Reusability: Previously stored information can be used again.
3. Large Data Storage: Files can store a large amount of information.
4. Data Sharing: Information can be accessed by different programs.
5. Record Management: Student, employee and customer records can be maintained easily.
6. Data Updating: Existing information can be modified by suitable file operations.

Important point: File handling provides permanent storage of data, whereas ordinary variables store data temporarily during program execution.

14. File Pointer and File Opening Modes

14.1 File Pointer

Definition

A file pointer is a special pointer used in C programming to access and manage a file. It is declared using the FILE data type provided by the standard input-output library. The file pointer maintains information required by the C library to perform file operations, such as reading and writing. It is used with functions such as fopen(), fclose(), fprintf(), fscanf() and fgetc().

Syntax

```
FILE *fp;
```

Here:

- FILE is a data type defined in stdio.h.
- * indicates that fp is a pointer.
- fp is the name of the file pointer.

Example

```
#include <stdio.h>

int main()
{
   FILE *fp;

   return 0;
}
```

Here, fp is a file pointer that can be used to access a file.

How File Pointer Works

Suppose we want to open a file named student.txt.

```
FILE *fp;

fp = fopen("student.txt", "r");
```

The fopen() function opens the file and returns a file pointer. The returned pointer is stored in fp. We then use this pointer to read data from the file.

Working of a file pointer

C Program

Requests to open student.txt

*FILE fp

Stores the file access information returned by fopen()

student.txt

File opened for reading

14.2 File Opening Modes

Definition

File opening modes specify how a file should be accessed when it is opened using the fopen() function. They determine whether the program can read data, write data or perform both operations. Some modes create a new file if it does not exist, while others require the file to exist already. Selecting the correct mode is important because some modes can erase the existing contents of a file.

Different File Opening Modes

Mode	Meaning	Description
"r"	Read	Opens an existing file for reading.
"w"	Write	Opens a file for writing; creates it if needed and truncates it if it exists.
"a"	Append	Opens a file for adding data at the end; creates it if needed.
"r+"	Read and Write	Opens an existing file for both reading and writing.
"w+"	Read and Write	Opens for reading and writing; creates or truncates the file.
"a+"	Read and Append	Opens for reading and appending; creates it if needed.

Explanation of Important Modes

A. Read Mode ("r")

Read mode is used to open an existing file for reading its contents. The file must already exist in the specified location. If the file does not exist, fopen() returns NULL. This mode does not allow the program to write new data into the file.

Example:

```
fp = fopen("student.txt", "r");
```

B. Write Mode ("w")

Write mode is used to create a new file or write data into an existing file. If the file does not exist, it is created automatically. If the file already exists, its previous contents are removed before writing new data. Therefore, this mode should be used carefully when existing information must be preserved.

Example:

```
fp = fopen("student.txt", "w");
```

C. Append Mode ("a")

Append mode is used to add new data at the end of an existing file. If the file does not exist, it is created automatically. Existing contents are preserved, and new data is added after the previous contents. This mode is useful when adding new student or employee records without deleting old records.

Example:

```
fp = fopen("student.txt", "a");
```

D. Read and Write Mode ("r+")

This mode allows both reading and writing in an existing file. The file must already exist, otherwise the opening operation fails. It does not automatically erase the existing contents. The program can perform reading and writing operations, subject to the file-position and update-stream rules.

Example:

```
fp = fopen("student.txt", "r+");
```

E. Write and Read Mode ("w+")

This mode allows both reading and writing in a file. If the file does not exist, it is created automatically. If the file already exists, its previous contents are removed. It is useful when a program needs to create a fresh file and then read and write data in it.

Example:

```
fp = fopen("student.txt", "w+");
```

F. Append and Read Mode ("a+")

This mode allows reading from a file and appending new data at the end. If the file does not exist, it is created automatically. Existing data is preserved, and all writes are directed to the end of the file. It is useful when a program needs to view existing records and add new records.

Example:

```
fp = fopen("student.txt", "a+");
```

Important Points

- "r" requires an existing file.
- "w" creates or clears a file.
- "a" preserves existing data and adds new data at the end.
- "r+" allows reading and writing without truncating the file.
- "w+" allows reading and writing but truncates existing contents.
- "a+" allows reading and appending.

15. Opening and Closing Text Files Using fopen() and fclose()

15.1 fopen() Function

Definition

fopen() is a standard library function used to open an existing file or create a new file in C programming. It accepts two arguments: the file name and the opening mode. The opening mode specifies whether the file should be opened for reading, writing or appending. If the operation is successful, the function returns a file pointer; otherwise, it returns NULL.

Syntax

```
FILE *fp;

fp = fopen("filename", "mode");
```

Example:

```
FILE *fp;

fp = fopen("student.txt", "w");
```

Explanation:

- FILE *fp; declares a file pointer.
- "student.txt" is the name of the file.
- "w" opens the file in write mode.
- fopen() returns a pointer to the opened file or NULL if it fails.

15.2 fclose() Function

Definition

fclose() is a standard library function used to close a file that has been opened using fopen(). Closing a file ensures that pending output is flushed to the file and that the associated resources are released. It is important to close files after completing reading or writing operations. The function returns 0 when the file is closed successfully and EOF if an error occurs.

Syntax

```
fclose(file_pointer);
```

Example:

```
fclose(fp);
```

Here, fp is the file pointer associated with the opened file.

Complete C Program

Program to create a text file and write data using fopen() and fclose().

```
#include <stdio.h>

int main()
{
   FILE *fp;

   fp = fopen("student.txt", "w");

   if(fp == NULL)
   {
      printf("Unable to open file");
      return 1;
   }

   fprintf(fp, "Welcome to C Programming\n");
   fprintf(fp, "File Handling Example");
```

```
    fclose(fp);

    printf("Data written successfully");

    return 0;
}
```

Output

```
Data written successfully
```

File created: student.txt

Contents of student.txt:

```
Welcome to C Programming
File Handling Example
```

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output header file containing file handling functions.
2. `FILE *fp;` declares a file pointer.
3. `fopen("student.txt", "w");` opens the file in write mode. If the file does not exist, it is created.
4. `if(fp == NULL)` checks whether the file was opened successfully.
5. If the file opening fails, the program displays an error message and returns 1.
6. `fprintf(fp, "Welcome to C Programming\n");` writes the first line into the file.
7. `fprintf(fp, "File Handling Example");` writes the second line into the file.
8. `fclose(fp);` closes the file after writing the data.
9. `printf()` displays a success message on the screen.
10. `return 0;` terminates the program successfully.

Important Points

- `fopen()` opens or creates a file.
- `fclose()` closes an opened file.
- Always check whether `fopen()` returned `NULL`.
- Always close a file after completing its operations.
- The `"w"` mode removes old contents if the file already exists.

16. Character and String File Operations

C provides functions to read and write individual characters and strings in text files. These functions are useful when a program needs to store or retrieve simple text information. Character operations work with one character at a time, whereas string operations work with a sequence of characters. The important functions in this category are `fgetc()`, `fputc()`, `fgets()` and `fputs()`.

16.1 `fputc()` Function

Definition

`fputc()` is a standard library function used to write a single character into a file. It accepts a character and a file pointer as arguments. The specified character is written at the current file position, and the file position advances after writing. It is useful when writing characters individually into a text file.

Syntax:

```
fputc(character, file_pointer);
```

Example:

```
fputc('A', fp);
```

Here, the character A is written into the file associated with fp.

C Program

Program to write individual characters into a file using fputc().

```
#include <stdio.h>

int main()
{
   FILE *fp;

   fp = fopen("character.txt", "w");

   if(fp == NULL)
   {
      printf("File opening failed");
      return 1;
   }

   fputc('H', fp);
   fputc('E', fp);
   fputc('L', fp);
   fputc('L', fp);
   fputc('O', fp);

   fclose(fp);

   printf("Characters written successfully");

   return 0;
}
```

Output:

```
Characters written successfully
```

Contents of character.txt:

```
HELLO
```

Explanation:

1. FILE *fp; declares a file pointer.
2. fopen("character.txt", "w"); opens a file in write mode.
3. if(fp == NULL) checks whether the file opening was successful.
4. fputc('H', fp); writes the character H into the file.
5. The remaining fputc() statements write E, L, L and O.
6. fclose(fp); closes the file.
7. printf() displays a success message.

16.2 fgetc() Function

Definition

fgetc() is a standard library function used to read one character at a time from a file. It reads the character at the current file position and moves the file position to the next character. The function returns the character as an int, not a char, so that it can also represent the special value EOF. It is useful for reading and processing text files character by character.

Syntax:

```
ch = fgetc(file_pointer);
```

Example:

```
ch = fgetc(fp);
```

Here, one character is read from the file associated with fp.

C Program

Program to read characters from a file using fgetc().

```
#include <stdio.h>

int main()
{
   FILE *fp;
   int ch;

   fp = fopen("character.txt", "r");

   if(fp == NULL)
   {
      printf("File not found");
      return 1;
   }

   printf("File contents:\n");

   while((ch = fgetc(fp)) != EOF)
   {
      putchar(ch);
   }

   fclose(fp);

   return 0;
}
```

Output

```
File contents:
HELLO
```

Explanation of the Program

1. FILE *fp; declares a file pointer.
2. int ch; declares an integer variable to store the character returned by fgetc().
3. fopen("character.txt", "r"); opens the file in read mode.
4. if(fp == NULL) checks whether the file exists and can be opened.
5. while((ch = fgetc(fp)) != EOF) reads one character at a time until the end of the file is reached.
6. putchar(ch); displays the character on the screen.
7. fclose(fp); closes the file.
8. return 0; terminates the program.

Important: Use an int variable for fgetc() so that every possible byte value can be distinguished from EOF.

16.3 fputs() Function

Definition

fputs() is a standard library function used to write a string into a file. It accepts a string and a file pointer as arguments. The complete string is written to the file until the terminating null character is encountered. Unlike puts(), the fputs() function does not automatically add a newline character at the end of the string.

Syntax:

```
fputs(string, file_pointer);
```

Example:

```
fputs("Welcome to C Programming", fp);
```

Here, the complete string is written into the file.

C Program

Program to write a string into a file using fputs().

```
#include <stdio.h>

int main()
{
   FILE *fp;

   fp = fopen("message.txt", "w");

   if(fp == NULL)
   {
      printf("File opening failed");
      return 1;
   }

   fputs("Welcome to C Programming\n", fp);
   fputs("Learning File Handling", fp);

   fclose(fp);

   printf("String written successfully");

   return 0;
}
```

Output

String written successfully

Contents of message.txt:

Welcome to C Programming
Learning File Handling

Explanation of the Program

1. FILE *fp; declares a file pointer.
2. fopen("message.txt", "w"); opens the file in write mode.
3. if(fp == NULL) checks whether the file was opened successfully.
4. fputs("Welcome to C Programming\n", fp); writes the first string into the file.
5. fputs("Learning File Handling", fp); writes the second string.
6. \n moves the next text to a new line.
7. fclose(fp); closes the file.
8. printf() displays a success message.

16.4 fgets() Function

Definition

fgets() is a standard library function used to read a string or a line of text from a file. It accepts three arguments: a character array, the maximum number of characters to read and the file pointer. It reads characters until a newline is encountered, the specified limit is reached or the end of the file occurs. When it reads a newline, that newline is retained in the character array if there is space for it, and the function adds a terminating null character.

Syntax:

```
fgets(string, size, file_pointer);
```

Example:

```
char str[100];

fgets(str, 100, fp);
```

Explanation:

- str is the character array used to store the input.
- 100 specifies the maximum buffer size, including space for the terminating null character.
- fp is the file pointer.

C Program

Program to read a string from a file using fgets().

```
#include <stdio.h>

int main()
{
   FILE *fp;
   char str[100];

   fp = fopen("message.txt", "r");

   if(fp == NULL)
   {
      printf("File not found");
      return 1;
   }

   printf("File contents:\n");

   while(fgets(str, sizeof(str), fp) != NULL)
   {
      printf("%s", str);
   }

   fclose(fp);

   return 0;
}
```

Output

```
File contents:
Welcome to C Programming
Learning File Handling
```

Explanation of the Program

1. `FILE *fp;` declares a file pointer.
2. `char str[100];` declares a character array to store a line read from the file.
3. `fopen("message.txt", "r");` opens the file in read mode.
4. `if(fp == NULL)` checks whether the file was opened successfully.
5. `while(fgets(str, sizeof(str), fp) != NULL)` reads lines from the file until no more input can be read.
6. `sizeof(str)` gives the total capacity of the character array, which is 100 bytes in this example.
7. `printf("%s", str);` displays the string read from the file. It does not add another newline automatically.
8. `fclose(fp);` closes the file.
9. `return 0;` terminates the program successfully.

Difference Between Character and String File Functions

Function	Operation	Purpose
fputc()	Write	Writes one character into a file.
fgetc()	Read	Reads one character from a file.
fputs()	Write	Writes a string into a file.
fgets()	Read	Reads a line or limited number of characters from a file.

Important Points

- fputc() writes one character.
- fgetc() reads one character and returns an int.
- fputs() writes a string without automatically adding a newline.
- fgets() reads a bounded amount of text and adds a terminating '\0' when successful.
- fgetc() returns EOF when it reaches the end of a file or encounters a read error.
- fgets() returns NULL when it cannot read another string.

17. Formatted File Operations Using fprintf() and fscanf()

Formatted file operations are used to write and read different types of data in a text file in a structured format. They work similarly to printf() and scanf(), but instead of using the keyboard and screen, they use a file pointer. The fprintf() function writes formatted data into a file, whereas fscanf() reads formatted data from a file. These functions are useful for storing and retrieving student records, employee details and other structured information.

17.1 fprintf() Function

Definition

fprintf() is a standard library function used to write formatted data into a file. It accepts a file pointer, a format string and the values that need to be written. It supports format specifiers such as %d, %f, %c and %s to write different data types. It is useful when multiple values need to be stored in a text file in an organized format.

Syntax:

```
fprintf(file_pointer, "format_string", variables);
```

Example:

```
int rollno = 101;
char name[] = "Ravi";
float marks = 85.5;

fprintf(fp, "%d %s %.2f", rollno, name, marks);
```

Explanation:

- fp is the file pointer.
- %d writes an integer.
- %s writes a string.
- %.2f writes a floating-point value with two digits after the decimal point.
- The values are written into the file in the specified order.

C Program

Program to write formatted student details into a file using fprintf().

```
#include <stdio.h>

int main()
{
   FILE *fp;
```

```
    int rollno = 101;
    char name[] = "Ravi";
    float marks = 85.5;

    fp = fopen("student.txt", "w");

    if(fp == NULL)
    {
        printf("File opening failed");
        return 1;
    }

    fprintf(fp, "%d %s %.2f\n", rollno, name, marks);

    fclose(fp);

    printf("Student details written successfully");

    return 0;
}
```

Output

Student details written successfully

Contents of student.txt:

101 Ravi 85.50

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output and file handling functions.
2. `FILE *fp;` declares a file pointer to access the file.
3. `int rollno = 101;` stores the student's roll number.
4. `char name[] = "Ravi";` stores the student's name.
5. `float marks = 85.5;` stores the student's marks.
6. `fopen("student.txt", "w");` opens the file in write mode.
7. `if(fp == NULL)` checks whether the file was opened successfully.
8. `fprintf(fp, "%d %s %.2f\n", rollno, name, marks);` writes all three values into the file in formatted form.
9. `fclose(fp);` closes the file.
10. `printf()` displays a success message.
11. `return 0;` terminates the program successfully.

17.2 fscanf() Function

Definition

fscanf() is a standard library function used to read formatted data from a file. It reads information according to the format specifiers provided in the function call. It can read different types of values such as integers, floating-point numbers and strings from a text file. The values read from the file are stored in the variables whose addresses are passed to the function.

Syntax:

```
fscanf(file_pointer, "format_string", &variables);
```

Example:

```
int rollno;
char name[30];
float marks;
fscanf(fp, "%d %29s %f", &rollno, name, &marks);
```

Explanation:

- %d reads an integer.
- %29s reads a string of at most 29 characters into name.
- %f reads a floating-point value.
- &rollno passes the address of the integer variable.
- name is already an array and is converted to a pointer to its first character.

C Program

Program to read student details from a file using fscanf().

```
#include <stdio.h>

int main()
{
   FILE *fp;

   int rollno;
   char name[30];
   float marks;

   fp = fopen("student.txt", "r");

   if(fp == NULL)
   {
      printf("File not found");
      return 1;
   }

   if(fscanf(fp, "%d %29s %f", &rollno, name, &marks) == 3)
   {
      printf("Student Details\n");

      printf("Roll Number = %d\n", rollno);
      printf("Name = %s\n", name);
      printf("Marks = %.2f\n", marks);
   }
   else
   {
      printf("Invalid file data");
   }

   fclose(fp);

   return 0;
}
```

Output

```
Student Details
Roll Number = 101
Name = Ravi
Marks = 85.50
```

Explanation of the Program

1. FILE *fp; declares a file pointer.
2. int rollno; declares a variable to store the roll number.
3. char name[30]; declares a character array to store the student's name.
4. float marks; declares a variable to store the marks.
5. fopen("student.txt", "r"); opens the file in read mode.
6. if(fp == NULL) checks whether the file opening was successful.
7. fscanf(fp, "%d %29s %f", &rollno, name, &marks) reads the three values from the file.
8. == 3 checks whether all three expected values were successfully read.
9. printf() displays the student details.
10. fclose(fp); closes the file.
11. return 0; terminates the program successfully.

Difference Between fprintf() and fscanf()

fprintf()	fscanf()
Writes formatted data into a file.	Reads formatted data from a file.
Used for output operations.	Used for input operations.
Similar to printf().	Similar to scanf().
Uses a file pointer and values.	Uses a file pointer and variable addresses.
Example: fprintf(fp, "%d", a);	Example: fscanf(fp, "%d", &a);

Important point: fprintf() writes formatted data into a file, whereas fscanf() reads formatted data from a file.

18. End-of-File Detection and Error Checking

18.1 End-of-File Detection

Definition

End-of-file detection is the process of identifying when a program has reached the end of a file while reading its contents. When a reading function tries to read beyond the available data, it indicates that no more input is available. In C, the EOF macro represents an end-of-file or input failure indication returned by certain input functions. End-of-file detection is useful when the number of records stored in a file is not known in advance.

What is EOF?

EOF stands for End Of File.

It is a special integer constant defined in the stdio.h header file.

It is commonly used with functions such as:

- fgetc()
- getc()
- fscanf()

Syntax

```
while((ch = fgetc(fp)) != EOF)
{
   // Process character
}
```

How EOF Works

Suppose a file contains:

```
HELLO
```

When the program reads the file:

1. First, it reads H.
2. Next, it reads E.
3. Then, it reads L.
4. Next, it reads L.
5. Then, it reads O.
6. After the last character, another read attempt encounters the end of the file and returns EOF.

C Program

Program to read a file character by character until EOF.

```
#include <stdio.h>

int main()
{
   FILE *fp;
   int ch;

   fp = fopen("message.txt", "r");

   if(fp == NULL)
   {
      printf("File not found");
      return 1;
   }

   printf("File contents:\n");

   while((ch = fgetc(fp)) != EOF)
   {
      putchar(ch);
   }

   fclose(fp);

   return 0;
}
```

Output

Suppose message.txt contains:

```
Welcome to C Programming
File Handling
```

The output will be:

```
File contents:
Welcome to C Programming
File Handling
```

Explanation of the Program

1. `FILE *fp;` declares a file pointer.
2. `int ch;` declares an integer variable to store the character returned by `fgetc()`.
3. `fopen("message.txt", "r");` opens the file in read mode.
4. `if(fp == NULL)` checks whether the file was opened successfully.
5. `while((ch = fgetc(fp)) != EOF)` reads characters one by one until the end of the file is reached.
6. `putchar(ch);` displays each character on the screen.
7. `fclose(fp);` closes the file.
8. `return 0;` terminates the program.

Important Note: `feof()` Function

C also provides the `feof()` function to check whether the end-of-file indicator has been set for a file.

Syntax:

```
feof(file_pointer);
```

It returns a nonzero value if the end-of-file indicator is set; otherwise, it returns zero.

Example:

```
if(feof(fp))
{
   printf("End of file reached");
}
```

Important: `feof()` becomes true only after a reading operation attempts to go beyond the available data. It should not be used as the condition for a loop that reads a file. Prefer checking the return value of `fgetc()`, `fgets()` or `fscanf()`.

18.2 Error Checking in File Handling

Definition

Error checking is the process of identifying whether a file operation has been performed successfully or has failed. File operations may fail because a file does not exist, the program does not have permission to access it or a storage-related problem occurs. C provides functions and return values to detect such problems. Error checking helps prevent incorrect results and allows programs to display meaningful error messages.

Common File Handling Errors

1. The requested file does not exist.
2. The program does not have permission to open the file.
3. The file cannot be created because of a storage or permission problem.
4. An error occurs while reading or writing data.
5. The disk or storage device becomes unavailable.

Important Error Checking Functions

A. Checking fopen() Using NULL

When fopen() fails to open a file, it returns NULL. We should always check the returned pointer before performing file operations.

Example:

```
fp = fopen("student.txt", "r");

if(fp == NULL)
{
   printf("File opening failed");
   return 1;
}
```

B. ferror() Function

ferror() is used to check whether a file error indicator has been set for a particular file. It returns a nonzero value if a reading or writing error has occurred. It returns zero when no file error has been recorded. It is useful for identifying errors during file operations.

Syntax:

```
ferror(file_pointer);
```

Example:

```
if(ferror(fp))
{
   printf("File reading error");
}
```

C. perror() Function

perror() is used to display a descriptive error message related to the most recently recorded system error. It prints the message supplied by the programmer followed by an explanation of the error. It is commonly used when a file cannot be opened or created. It helps programmers understand why a file operation failed.

Syntax:

```
perror("Error message");
```

Example:

```
fp = fopen("student.txt", "r");

if(fp == NULL)
{
   perror("Unable to open file");
   return 1;
}
```

D. Checking fclose() Return Value

fclose() returns 0 when the file is closed successfully and EOF when an error occurs during closing.

Example:

```
if(fclose(fp) == EOF)
{
   printf("File closing error");
}
```

C Program: Program to demonstrate file error checking.

```
#include <stdio.h>

int main()
{
   FILE *fp;
   int ch;

   fp = fopen("student.txt", "r");

   if(fp == NULL)
   {
      perror("Unable to open file");
      return 1;
   }

   while((ch = fgetc(fp)) != EOF)
   {
      putchar(ch);
   }

   if(ferror(fp))
   {
      printf("\nError occurred while reading file");
   }

   if(fclose(fp) == EOF)
   {
      printf("Error closing file");
      return 1;
   }

   return 0;
}
```

Explanation of the Program

1. FILE *fp; declares a file pointer.
2. int ch; declares a variable to store characters read from the file.
3. fopen("student.txt", "r"); attempts to open the file.
4. if(fp == NULL) checks whether the file opening failed.
5. perror() displays a descriptive error message if opening fails.
6. The while loop reads the file character by character.
7. putchar(ch); displays each character.
8. ferror(fp) checks whether a file reading error occurred.
9. fclose(fp) closes the file and its return value is checked.
10. return 0; indicates successful program termination.

Difference Between EOF and ferror()

EOF	ferror()
Indicates that the end-of-file indicator has been set.	Checks whether the file error indicator has been set.
Commonly used with reading functions.	Used to detect reading or writing errors.
It is a macro.	It is a standard library function.
Example: ch != EOF	Example: ferror(fp)

Important points:

- Always check whether fopen() returns NULL.
- Use EOF to detect the end of input from functions such as fgetc().
- Use ferror() to detect file reading or writing errors.
- Use perror() to display a descriptive error message.
- Check the return value of fclose() when file-writing errors matter.

19. Writing, Reading, Displaying and Searching Student Records in a Text File

This topic combines structures and file handling to create a simple student record management program. It demonstrates how student details can be stored permanently in a text file and retrieved later. We will use a structure to represent each student and file functions to write, read, display and search the records. This is an important practical application of structures and file handling in C programming.

19.1 Writing Student Records into a File

Definition

Writing student records means storing student information in a text file using a C program. The program accepts student details such as roll number, name and marks from the user. These details are written into a file using functions such as fprintf(). Once the data is written and the file is closed successfully, the records remain stored for future use.

C Program

Program to write student records into a text file.

```
#include <stdio.h>

struct Student
{
   int rollno;
   char name[30];
   float marks;
};

int main()
{
   FILE *fp;
   struct Student s;
   int n, i;

   fp = fopen("students.txt", "w");

   if(fp == NULL)
```

```
    {
        printf("File opening failed");
        return 1;
    }

    printf("Enter number of students: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nEnter roll number: ");
        scanf("%d", &s.rollno);

        printf("Enter name: ");
        scanf("%29s", s.name);

        printf("Enter marks: ");
        scanf("%f", &s.marks);

        fprintf(fp, "%d %s %.2f\n",
                s.rollno, s.name, s.marks);
    }

    fclose(fp);

    printf("\nStudent records saved successfully");

    return 0;
}
```

Output

```
Enter number of students: 2

Enter roll number: 101
Enter name: Ravi
Enter marks: 85

Enter roll number: 102
Enter name: Priya
Enter marks: 90

Student records saved successfully
```

Contents of students.txt:

```
101 Ravi 85.00
102 Priya 90.00
```

Explanation of the Program

1. `#include <stdio.h>` includes standard input-output and file handling functions.
2. `struct Student` declares a structure containing roll number, name and marks.
3. `FILE *fp;` declares a file pointer.
4. `struct Student s;` declares a structure variable to hold one student's details at a time.
5. `int n, i;` declares variables for the number of students and loop counter.
6. `fopen("students.txt", "w");` opens the file in write mode. If it already exists, its previous contents are removed.
7. `if(fp == NULL)` checks whether the file was opened successfully.
8. `scanf("%d", &n);` accepts the number of student records.
9. The `for` loop executes once for each student.

10. scanf("%d", &s.rollno); accepts the student's roll number.
11. scanf("%29s", s.name); accepts the student's name, limited to 29 characters.
12. scanf("%f", &s.marks); accepts the student's marks.
13. fprintf() writes all the details into the file in a formatted manner.
14. fclose(fp); closes the file after writing all records.
15. printf() displays a confirmation message.

19.2 Reading Student Records from a File

Definition

Reading student records means retrieving previously stored information from a text file. The program opens the file in read mode and uses file reading functions to retrieve the stored values. These values can be stored in variables or structure members for further processing. Reading records allows the program to use previously saved information without asking the user to enter the same details again.

C Program

Program to read student records from a text file.

```
#include <stdio.h>

struct Student
{
   int rollno;
   char name[30];
   float marks;
};

int main()
{
   FILE *fp;
   struct Student s;

   fp = fopen("students.txt", "r");

   if(fp == NULL)
   {
      printf("File not found");
      return 1;
   }

   printf("Student Records\n");

   while(fscanf(fp, "%d %29s %f",
            &s.rollno, s.name, &s.marks) == 3)
   {
      printf("\nRoll Number = %d", s.rollno);
      printf("\nName = %s", s.name);
      printf("\nMarks = %.2f\n", s.marks);
   }

   if(ferror(fp))
   {
      printf("\nError reading file");
   }

   fclose(fp);

   return 0;
}
```

Output

```
Student Records

Roll Number = 101
Name = Ravi
Marks = 85.00

Roll Number = 102
Name = Priya
Marks = 90.00
```

Explanation of the Program

1. `struct Student` declares a structure for storing student information.
2. `FILE *fp;` declares a file pointer.
3. `struct Student s;` declares a structure variable to receive the records.
4. `fopen("students.txt", "r");` opens the file in read mode.
5. `if(fp == NULL)` checks whether the file exists and can be opened.
6. `while(fscanf(fp, "%d %29s %f", &s.rollno, s.name, &s.marks) == 3)` reads one complete student record at a time.
7. The condition `== 3` ensures that all three expected fields were successfully read.
8. `printf()` displays the roll number, name and marks.
9. The loop repeats until no more complete records can be read.
10. `ferror(fp)` checks whether a file reading error occurred.
11. `fclose(fp);` closes the file.
12. `return 0;` terminates the program successfully.

19.3 Displaying All Student Records

Definition

Displaying student records means showing all the information stored in a file on the computer screen in an organized format. The program opens the file in read mode and retrieves each record one by one. The retrieved details are displayed using the `printf()` function. This operation helps users view all the student information stored in the file.

C Program

Program to display all student records in table format.

```
#include <stdio.h>

struct Student
{
   int rollno;
   char name[30];
   float marks;
};

int main()
{
   FILE *fp;
   struct Student s;

   fp = fopen("students.txt", "r");

   if(fp == NULL)
```

```
    {
        printf("File not found");
        return 1;
    }

    printf("\n-----------------------------------\n");
    printf("Roll No\tName\tMarks\n");
    printf("-----------------------------------\n");

    while(fscanf(fp, "%d %29s %f",
                &s.rollno, s.name, &s.marks) == 3)
    {
        printf("%d\t%s\t%.2f\n",
              s.rollno, s.name, s.marks);
    }

    fclose(fp);

    return 0;
}
```

Output

```
-----------------------------------
Roll No Name    Marks
-----------------------------------
101     Ravi    85.00
102     Priya   90.00
```

Explanation of the Program

1. `struct Student` declares the structure used to store student information.
2. `FILE *fp;` declares a file pointer.
3. `struct Student s;` declares a variable to receive each student record.
4. `fopen("students.txt", "r");` opens the student file for reading.
5. `if(fp == NULL)` checks whether the file opening was successful.
6. The first `printf()` statements display the table heading.
7. The `while` loop reads one student record at a time using `fscanf()`.
8. `printf("%d\t%s\t%.2f\n", ...)` displays the record in a table-like format.
9. The loop continues until no more complete records are available.
10. `fclose(fp);` closes the file.
11. `return 0;` terminates the program.

19.4 Searching Student Records in a File

Definition

Searching student records means finding a particular student's information from the records stored in a file. The user provides a value such as a roll number that is used as the search key. The program reads each record from the file and compares its roll number with the entered value. If a matching record is found, the program displays that student's details; otherwise, it displays a message indicating that the record was not found.

Example

Suppose the file contains:

Roll Number	Name	Marks
101	Ravi	85
102	Priya	90
103	Kumar	78

If the user searches for roll number 102, the program should display:

```
Student Found

Roll Number = 102
Name = Priya
Marks = 90.00
```

C Program

Program to search for a student using roll number in a text file.

```
#include <stdio.h>

struct Student
{
   int rollno;
   char name[30];
   float marks;
};

int main()
{
   FILE *fp;
   struct Student s;
   int search, found = 0;

   fp = fopen("students.txt", "r");

   if(fp == NULL)
   {
      printf("File not found");
      return 1;
   }

   printf("Enter roll number to search: ");
   scanf("%d", &search);

   while(fscanf(fp, "%d %29s %f",
            &s.rollno, s.name, &s.marks) == 3)
   {
      if(s.rollno == search)
      {
         printf("\nStudent Found\n");

         printf("Roll Number = %d\n", s.rollno);
         printf("Name = %s\n", s.name);
         printf("Marks = %.2f\n", s.marks);

         found = 1;
         break;
      }
```

```
    }

    if(found == 0)
    {
        printf("Student record not found");
    }

    fclose(fp);

    return 0;
}
```

Output 1: Record Found

```
Enter roll number to search: 102

Student Found
Roll Number = 102
Name = Priya
Marks = 90.00
```

Output 2: Record Not Found

```
Enter roll number to search: 105
Student record not found
```

Explanation of the Program

1. `struct Student` declares a structure containing roll number, name and marks.
2. `FILE *fp;` declares a file pointer.
3. `struct Student s;` declares a structure variable to store the record currently being read.
4. `int search, found = 0;` declares the search variable and initializes the `found` flag to zero.
5. `fopen("students.txt", "r");` opens the file in read mode.
6. `if(fp == NULL)` checks whether the file opening was successful.
7. `scanf("%d", &search);` accepts the roll number that the user wants to search for.
8. The `while` loop reads student records one by one using `fscanf()`.
9. `if(s.rollno == search)` compares the current record's roll number with the entered search value.
10. If the values match, the program displays the student's details.
11. `found = 1;` indicates that the record has been found.
12. `break;` stops the loop because the required record has been located.
13. `if(found == 0)` checks whether no matching record was found.
14. `fclose(fp);` closes the file after the search operation.
15. `return 0;` terminates the program successfully.

Important point: The program uses linear searching because it checks student records one by one until the required roll number is found.